

Block 1: Selecting Specific Columns

Concept: To optimize queries and see only relevant information, we select columns by name instead of using *.

Query Structure:

```
SELECT column1, column2, ... FROM table_name;
```

Practical Activity 1:

1. Select only the first name, last name, and salary of all employees.

```
SELECT first_name, last_name, salary FROM employees;
```
 2. **Exercise:** Now, select the department name and location ID from the departments table.

```
SELECT department_name, location_id FROM departments;
```
-

Block 2: Filtering Data with WHERE

Concept: The WHERE clause is used to filter records and return only those that satisfy a specific condition. We'll use comparison operators: =, != (or <>), >, <, >=, <=. (Remember: text goes in single quotes ' ').

Query Structure:

```
SELECT columns FROM table_name WHERE condition;
```

Practical Activity 2:

1. Find all employees who work in department ID 90 (Executive).

```
SELECT first_name, last_name, department_id FROM employees WHERE department_id = 90;
```
 2. Find all employees whose last name is exactly 'King'.

```
SELECT * FROM employees WHERE last_name = 'King';
```
 3. **Exercise:** List the employees who earn a salary **greater than 10,000**.

```
SELECT first_name, last_name, salary FROM employees WHERE salary > 10000;
```
-

Block 3: Combining Filters with AND and OR

Concept:

- AND: Returns records that satisfy **all** conditions.
- OR: Returns records that satisfy **at least one** of the conditions.

Query Structure:

```
SELECT columns FROM table_name WHERE condition1 AND condition2;
```

```
SELECT columns FROM table_name WHERE condition1 OR condition2;
```

Practical Activity 3:

1. List the employees from department 60 **AND** who have a salary greater than 5000.

```
SELECT first_name, last_name, salary FROM employees WHERE department_id = 60 AND salary > 5000;
```
2. **Exercise:** List the employees who are from department 50 **OR** department 80.

```
SELECT first_name, last_name, department_id FROM employees WHERE department_id = 50 OR
department_id = 80;
```

Block 4: Advanced Filtering Operators (IN, BETWEEN, LIKE)

Concept: More efficient and powerful ways to filter.

- **IN:** Simplifies multiple OR conditions for a list of values.
- **BETWEEN:** Checks if a value is within a range (inclusive).
- **LIKE:** Searches for patterns in text using wildcards (% for any sequence, _ for a single character).

Structure and Practical Activities 4:

1. **Using IN:** Rewrite the last exercise more cleanly.

-- Structure: WHERE column IN (value1, value2, ...)

```
SELECT first_name, last_name, department_id FROM employees WHERE department_id IN (50, 80);
```

2. **Using BETWEEN:** Find employees with a salary between 7000 and 9000.

-- Structure: WHERE column BETWEEN min_value AND max_value

```
SELECT first_name, salary FROM employees WHERE salary BETWEEN 7000 AND 9000;
```

3. **Using LIKE:** Find all employees whose first name starts with the letter 'J'.

-- Structure: WHERE column LIKE 'pattern'

```
SELECT first_name FROM employees WHERE first_name LIKE 'J%';
```

4. **Exercise:** Using the operators above, find employees whose job_id is IT_PROG or SA_REP **and** whose hire_date was in the year 2016. (*Hint: Oracle uses TO_DATE for dates, but for this exercise, we can use BETWEEN with the start and end dates of the year.*)

```
SELECT first_name, job_id, hire_date
FROM employees
WHERE job_id IN ('IT_PROG', 'SA_REP')
AND hire_date BETWEEN TO_DATE('2016-01-01', 'YYYY-MM-DD') AND TO_DATE('2016-12-31', 'YYYY-MM-DD');
```

Block 5: Sorting Results with ORDER BY

Concept: The ORDER BY clause organizes the result set based on one or more columns. The order can be ascending (ASC, default) or descending (DESC).

Query Structure:

```
SELECT columns FROM table_name [WHERE condition] ORDER BY sort_column [ASC|DESC];
```

Practical Activity 5:

1. List all employees, sorted by salary from highest to lowest.

```
SELECT first_name, last_name, salary FROM employees ORDER BY salary DESC;
```

Copier

2. **Exercise (Multi-column Sort):** List all employees, ordered first by department_id (ascending), and then within each department, by salary (descending).

```
SELECT first_name, last_name, department_id, salary FROM employees ORDER BY department_id ASC,  
salary DESC;
```

Block 6: JOINS

1. INNER JOIN: The Intersection

Concept: This is the most common type of join. It returns only the rows where the join condition is met in both tables. Think of it as the intersection in a Venn diagram.

Syntax:

```
SELECT  t1.column, t2.column  
FROM    table1 t1  
INNER JOIN table2 t2 ON t1.common_column = t2.common_column;
```

Practical Example: List all employees and the name of the department they work in.

```
SELECT  e.first_name, e.last_name, d.department_name  
FROM    employees e  
INNER JOIN departments d ON e.department_id = d.department_id;
```

Result Analysis: This query will only show employees who are assigned to a department that actually exists in the departments table. If an employee had a department_id that didn't exist, or was NULL, they would not appear in this list.

2. LEFT JOIN (or LEFT OUTER JOIN): All from the Left

Concept: This returns all rows from the left table (the one mentioned after FROM), and only the matching rows from the right table. If there is no match in the right table, the columns from the right table will contain NULL.

Syntax:

```
SELECT  t1.column, t2.column  
FROM    table1 t1  
LEFT JOIN table2 t2 ON t1.common_column = t2.common_column;
```

Practical Example: List ALL employees, and show their department name if they have one.

The HR schema has one employee, Kimberly Grant, whose department_id is NULL. She will not appear in an INNER JOIN, but she will here.

```
SELECT  e.first_name, e.last_name, d.department_name  
FROM    employees e  
LEFT JOIN departments d ON e.department_id = d.department_id;
```

Result Analysis: The list will contain every single employee. For Kimberly Grant, the department_name column will be NULL because her department_id does not match any row in the departments table.

3. RIGHT JOIN (or RIGHT OUTER JOIN): All from the Right

Concept: This is the inverse of a LEFT JOIN. It returns all rows from the right table (the one mentioned after JOIN), and only the matching rows from the left table. If there is no match in the left table, its columns will be NULL.

Syntax:

```
SELECT t1.column, t2.column
FROM table1 t1
RIGHT JOIN table2 t2 ON t1.common_column = t2.common_column;
```

Practical Example: List ALL departments and which employees work in them. If a department has no employees, it must still be on the list.

The HR schema has many departments with no employees (e.g., Treasury, Corporate Tax, etc.). They will not appear in an INNER JOIN.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id
ORDER BY d.department_name;
```

Result Analysis: The list will contain every department. For departments like "Treasury", the first_name and last_name columns will be NULL because no employee has their department_id.

4. FULL OUTER JOIN: All from Everywhere

Concept: This join combines the results of both LEFT and RIGHT joins. It returns all rows from both tables. It places NULLs where a match is not found on either side. Think of it as the complete union in a Venn diagram.

Syntax:

```
SELECT t1.column, t2.column
FROM table1 t1
FULL OUTER JOIN table2 t2 ON t1.common_column = t2.common_column;
```

Practical Example: Create a master list of all employees and all departments, matching them where possible.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.department_id
ORDER BY e.first_name;
```

Result Analysis: You will see Kimberly Grant with a NULL department name, and you will also see departments like "Treasury" with NULL employee names.

5. SELF JOIN: Joining a Table to Itself

Concept: This is not a new type of join, but a technique. You join a table to itself to create relationships between rows in the same table. This is essential for hierarchical data. The key is to use table aliases to treat the table as two separate entities.

Practical Example: For each employee, show their name and the name of their direct manager.

Both the employee and the manager are in the employees table. We link them using employee.manager_id = manager.employee_id.

```
SELECT worker.first_name || ' ' || worker.last_name AS employee_name,
       manager.first_name || ' ' || manager.last_name AS manager_name
FROM employees worker -- Alias 1: The employee
INNER JOIN employees manager ON worker.manager_id = manager.employee_id; -- Alias 2: The manager
```

Result Analysis: The query successfully links a row (the worker) to another row in the same table (the manager) based on the manager_id relationship. Steven King, the president, will not appear in this list because his manager_id is NULL, so the INNER JOIN fails.

6. CROSS JOIN: The Cartesian Product

Concept: This join returns every possible combination of rows from both tables. It is rarely used in standard business reporting, as it can generate massive result sets. No ON clause is used.

Syntax:

```
SELECT  t1.column, t2.column
FROM    table1 t1
CROSS JOIN  table2 t2;
```

Practical Example: Create a list of every possible job assignment for every region (not a practical business request, but illustrates the concept).

```
SELECT  r.region_name, j.job_title
FROM    regions r
CROSS JOIN  jobs j;
```

Result Analysis: If there are 4 regions and 19 jobs, the result will be $4 * 19 = 76$ rows, showing every combination of region and job title.

Summary and Practice Exercises

JOIN Type	Purpose
INNER JOIN	Returns only rows that have a match in both tables.
LEFT JOIN	Returns all rows from the left table, and matching rows from the right table (NULLs on the right if no match).
RIGHT JOIN	Returns all rows from the right table, and matching rows from the left table (NULLs on the left if no match).
FULL OUTER JOIN	Returns all rows from both tables, with NULLs where matches don't exist in either table.
SELF JOIN	(A technique) Joins a table to itself to relate rows to other rows in the same table (e.g., hierarchies).
CROSS JOIN	Returns every possible combination of rows from the two tables (Cartesian product).

Exercises:

1. **INNER JOIN Challenge:** Write a query to display the first name of an employee, their job title, and the city where their department is located. (Hint: This will require joining employees, jobs, departments, and locations).
2. **LEFT JOIN Challenge:** List the names of all countries and the number of departments located in each. If a country from the countries table has no departments, it should still appear on the list with a count of 0.
3. **SELF JOIN Challenge:** Find the names of all employees who were hired before their manager. (Hint: You will need a SELF JOIN on the employees table and a WHERE clause comparing hire dates).

Block 7: Inserting Data with INSERT

Concept: INSERT INTO is the command used to add new rows of data to a table.

Attention: The employees table has rules (constraints). employee_id must be unique. email, hire_date, and job_id cannot be null. We will use safe data to avoid errors.

Query Structure:

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

Practical Activity 7:

1. Let's insert a new fictional employee. (Note: *TO_DATE* is the Oracle function to convert a string to a date. Today's date is January 11, 2026).

```
INSERT INTO employees (employee_id, first_name, last_name, email, phone_number, hire_date, job_id, salary, department_id) VALUES (300, 'Maria', 'Souza', 'MSOUZA', '555.123.4567', TO_DATE('2026-01-11', 'YYYY-MM-DD'), 'IT_PROG', 7500, 60);
```

2. **Exercise (Verification):** Now, prove that the record was inserted. Write a SELECT query to find the employee 'Maria Souza' or employee_id 300.

```
SELECT * FROM employees WHERE employee_id = 300;
```

Test:

Business Scenario: The HR department needs to generate a list of employees for a strategic salary and role review. This list is not for everyone; it must include only two specific groups of employees who meet multiple criteria. After generating the list, a key new hire who was just brought on board needs to be added to the system and verified to see if they would also qualify for this same report.

Part A: Adding a New Employee (INSERT Command)

Your task: HR informs you that a new programmer was hired today (January 11, 2026), but is not yet in the system. Add the following employee to the employees table:

employee_id: 301
first_name: Fabio
last_name: Martins
email: fmartins@email.com
phone_number: 555.111.2222
hire_date: 11-JAN-26
job_id: IT_PROG
salary: 8200
department_id: 60

Write and execute the INSERT command to add this record.

Part B: Generating the Main Report (SELECT Query)

Your Task: Write a single SELECT query that returns a list of employees who meet exactly the following criteria:

Inclusion Criteria (the employee must meet ONE of the two conditions below):

1. Is a "Sales Representative" (job_id = 'SA_REP') with a salary between 7,500 and 10,000 (inclusive). OR
2. Is a "Programmer" (job_id = 'IT_PROG').

Additional Criterion (the employee must meet THIS condition in addition to one of the above):

3. The employee's first name (first_name) must contain the letter 'a' (lowercase), in any position.

Report Format Requirements:

- Columns to display: employee_id, first_name, last_name, job_id, salary.
- Sorting: The result must be ordered first by job_id alphabetically (A-Z) and then by salary in descending order (highest to lowest).

Hint: You will need to use parentheses () in your WHERE clause to correctly group the AND and OR logical conditions.

Part C: Final Verification**Your Task:**

1. After successfully executing the INSERT from Part B, rerun the complex SELECT query you created in Part A.
 2. Answer the following question: Does the new employee, Fabio Martins, appear on the new report? Why or why not? (The answer should be based on analyzing the criteria from Part A against the data inserted in Part B).
-

Solution and Explanation**Solution - Part A:**

```
SELECT employee_id, first_name, last_name, job_id, salary
FROM employees
WHERE ( (job_id = 'SA_REP' AND salary BETWEEN 7500 AND 10000) OR
       (job_id = 'IT_PROG') ) AND
       lower(first_name) LIKE '%a%'
ORDER BY job_id ASC, salary DESC;
```

Solution - Part B:

```
INSERT INTO employees (employee_id, first_name, last_name, email, phone_number, hire_date, job_id, salary,
department_id)VALUES (301, 'Fabio', 'Martins', 'FMARTINS', '555.111.2222', TO_DATE('2026-01-11', 'YYYY-MM-DD'),
'IT_PROG', 8200, 60);
```

Solution - Part C: After re-running the query from Part A, the answer to the question is:

"Yes, Fabio Martins appears on the report."

Why?

1. He satisfies one of the conditions inside the OR block: his job_id is 'IT_PROG'.
2. He also satisfies the AND condition: his first name 'Fabio' contains the letter 'a'.

DAY 2

Block 1: Advanced Filtering with WHERE (25 minutes)

Concept: Let's expand our filtering capabilities beyond the basics. We will introduce NOT to invert conditions and IS NULL to find missing data, a very common task.

Structures and Practical Activities:

1. **Using NOT:** The NOT operator can be used with IN, LIKE, BETWEEN, etc., to exclude results.

- **Task:** Find all employees who are **not** in departments 50, 60, or 80.

```
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id NOT IN (50, 60, 80);
```

2. **Handling NULL values:** NULL represents an unknown or missing value. You cannot use = or != to check for NULL. You must use IS NULL or IS NOT NULL.

- **Task:** Find the employee who does not report to any manager (i.e., their manager_id is null). This is usually the top-level president or CEO.

```
SELECT first_name, last_name, manager_id FROM employees WHERE manager_id IS NULL;
```

3. **Exercise:** Find all employees who **have** a commission percentage (commission_pct is not null) **and** their job_id is not SA_REP.

```
SELECT first_name, job_id, commission_pct
FROM employees
WHERE commission_pct IS NOT NULL AND job_id <> 'SA_REP';
```

Block 2: Introduction to Aggregate Functions (25 minutes)

Concept: Aggregate functions perform a calculation on a set of rows and return a single summary row. They are the foundation of data analysis in SQL.

Common Functions:

- COUNT(): Counts the number of rows.
- SUM(): Calculates the sum of a numeric column.
- AVG(): Calculates the average of a numeric column.
- MAX(): Returns the maximum value in a column.
- MIN(): Returns the minimum value in a column.

Practical Activities:

1. **Count:** How many employees work at the company? COUNT(*) counts all rows.

```
SELECT COUNT(*) FROM employees;
```

2. **Sum:** What is the company's total monthly salary payroll?

```
SELECT SUM(salary) FROM employees;
```

3. **Exercise:** Find the average salary, the highest salary, and the lowest salary of all employees in a single query. Let's give our results clearer names using column aliases.

```
SELECT AVG(salary) AS average_salary, MAX(salary) AS highest_salary, MIN(salary) AS lowest_salary
FROM employees;
```

Block 3: Grouping Data with GROUP BY (35 minutes)

Concept: GROUP BY is one of the most powerful clauses in SQL. It is used with aggregate functions to group rows that have the same values in specified columns into summary rows.

Key Rule: Any column in the SELECT list that is not an aggregate function **must** be in the GROUP BY clause.

Structure & Practical Activities:

1. **Task:** Count the number of employees *in each department*.

```
SELECT department_id, COUNT(*) AS number_of_employees
FROM employees
WHERE department_id IS NOT NULL -- Good practice to exclude nulls from this count
GROUP BY department_id
ORDER BY department_id;
```

2. **Task:** Calculate the average salary *for each job title*.

```
SELECT job_id, AVG(salary) AS avg_salary_for_job
FROM employees
GROUP BY job_id
ORDER BY avg_salary_for_job DESC;
```

3. **Exercise:** Find the total salary paid (SUM) for each department. Show only the department_id and the total salary. Order the results by the total salary from highest to lowest.

```
SELECT department_id, SUM(salary) AS total_payroll_by_dept
FROM employees
GROUP BY department_id
ORDER BY total_payroll_by_dept DESC;
```

Block 4: Filtering Groups with HAVING (30 minutes)

Concept: WHERE filters rows *before* aggregation. But what if you want to filter based on the result of an aggregate function? That's what HAVING is for. It filters groups *after* they have been created.

Structure:

```
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING [condition_on_aggregate_function];
```

Practical Activities:

1. **Task:** Show only the departments that have **more than 10 employees**.

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 10;
```

2. **Exercise:** List the job_ids where the average salary for that job is **greater than 9,000**. Order the result by the average salary.

```
SELECT job_id, AVG(salary) as average_salary
FROM employees
GROUP BY job_id
HAVING AVG(salary) > 9000
ORDER BY average_salary DESC;
```

Block 5: Power Up with Oracle's Built-in Functions (40 minutes)

Concept: Oracle provides a rich library of single-row functions that operate on each row, transforming the data. Let's explore some of the most common ones.

Types & Activities:

1. String Functions:

- UPPER(), LOWER(), INITCAP(): Change the case of text.
- SUBSTR(string, start_position, length): Extracts a substring.
- LENGTH(): Gets the length of a string.
- **Task:** Display employee first names in uppercase and their last names with only the first letter capitalized.

```
SELECT UPPER(first_name), INITCAP(last_name) FROM employees;
```

- **Exercise:** Show the first name and the first three letters of the last name for all employees in department 50.

```
SELECT first_name, SUBSTR(last_name, 1, 3) AS last_name_abbr
FROM employees
WHERE department_id = 50;
```

2. Numeric Functions:

- ROUND(number, decimal_places): Rounds a number.
- TRUNC(number, decimal_places): Truncates (cuts off) a number.
- **Task:** Show the monthly salary and the annual salary (salary * 12) for all employees, with the annual salary rounded to the nearest thousand.

```
SELECT first_name, salary, ROUND(salary * 12, -3) AS rounded_annual_salary
FROM employees;
```

3. Date Functions:

- SYSDATE: Returns the current database server date and time.
- MONTHS_BETWEEN(date1, date2): Returns the number of months between two dates.
- ADD_MONTHS(date, num_months): Adds a number of months to a date.
- **Exercise:** For each employee, calculate how many months they have been working at the company as of today (January 11, 2026). Show their name, hire date, and the calculated tenure in months, rounded to a whole number.

```
SELECT first_name, hire_date,
ROUND(MONTHS_BETWEEN(sysdate, hire_date)) AS tenure_in_months
FROM employees;
```

Block 6: Final Challenge & Summary (25 minutes)

Challenge Exercise: Let's combine everything. Write a single query to find:

- The job_ids where the maximum salary for that job is over 12,000.
- For these jobs, show the job_id and the number of employees (COUNT).
- Only include employees who were hired before the year 2016 in your calculation.

- Order the final result by the number of employees in descending order.

Solution:

```
SELECT job_id, COUNT(*) AS number_of_employees
FROM employees
WHERE hire_date < TO_DATE('2016-01-01', 'YYYY-MM-DD')
GROUP BY job_id
HAVING MAX(salary) > 12000
ORDER BY number_of_employees DESC;
```

Business Scenario: The Chief Financial Officer (CFO) has requested a high-level report to identify departments that require a budget review. The focus is on departments that might be either under-resourced or have a disproportionately high payroll cost relative to their size. Your task is to generate this report and then analyze its findings.

Part A: Generating the Departmental Analysis Report

Your Task: Write a single SELECT query that identifies "departments of interest". A department is considered "of interest" if it meets the specific criteria outlined below.

Inclusion Criteria (a department must meet ONE of the following two conditions):

1. It is a small department with a high payroll, defined as having **fewer than 5 employees** AND an average salary **greater than 8,000**. **OR**
2. It is a very large department, defined as having **more than 15 employees**.

Exclusion Criterion (an important filter to apply BEFORE grouping):

3. The analysis must **exclude** all employees from the 'Executive' department (department ID 90), as their budget is managed separately.

Report Format Requirements:

- **Columns to be displayed:**
 1. A custom "Department Code", created by concatenating the department ID with the text - REVIEW (e.g., 50-REVIEW).
 2. The total number of employees in that department, aliased as employee_count.
 3. The average salary for that department, rounded to two decimal places, aliased as average_salary.
 - **Data Integrity:** The report should ignore any employees who are not assigned to a department (department_id is null).
 - **Sorting:** The final report must be sorted by average_salary in descending order.
-

Part B: Critical Analysis of the Query Logic

Your Task: After running your query from Part A, answer the following conceptual questions without running any new queries.

1. **Question 1:** The Sales department (ID 80) has an average salary of approximately 8,955 and contains 34 employees. Based on your query's logic, will this department appear in your final report? Explain precisely which condition(s) it meets or fails in your HAVING clause.

2. **Question 2:** Consider the difference between the WHERE clause and the HAVING clause in your query. What would have happened if you had tried to put the condition `COUNT(*) > 15` in the WHERE clause instead of the HAVING clause? Why?
-

Part C: Impact of a Scenario Change

Your Task: Let's imagine the CFO changes their mind and now wants the Executive department (ID 90) to be included in the analysis.

1. Briefly describe the single change you would make to your query from Part A to accommodate this new request.
2. The Executive department (ID 90) has 3 employees and an average salary of 19,333. If you make the change from step 1, will this department now appear in the report? Justify your answer by walking through the logic of your HAVING clause for this specific department.

Solution and Explanation (for the instructor)

Solution - Part A:

```
SELECT
department_id || '-REVIEW' AS department_code, COUNT(*) AS employee_count, ROUND(AVG(salary), 2) AS
average_salary
FROM employees
WHERE department_id IS NOT NULL AND
      department_id <> 90 -- Exclusion Criterion applied BEFORE grouping
GROUP BY department_id
HAVING (COUNT(*) < 5 AND AVG(salary) > 8000) -- Condition 1 OR (COUNT(*) > 15) -- Condition 2
ORDER BY average_salary DESC;
```

- **Complexity:** This query combines WHERE (for pre-filtering) with GROUP BY and a complex, multi-part HAVING clause. It also uses aliasing, a numeric function (ROUND), and a string operation (|| concatenation), covering all major topics of Day 2.

Solution - Part B:

1. **Answer 1: "Yes, the Sales department (ID 80) will appear.** It fails the first condition in the HAVING clause because its employee_count (34) is not less than 5. However, it meets the second condition (`COUNT(*) > 15`) because 34 is greater than 15. Since the conditions are joined by OR, only one needs to be true."
2. **Answer 2: "It would cause an error.** The error would be ORA-00934: group function is not allowed here. This is because the WHERE clause operates on individual rows *before* they are grouped. Aggregate functions like `COUNT(*)` can only operate on groups of rows, so they are only valid in the SELECT list, the HAVING clause, or the ORDER BY clause."

Solution - Part C:

1. **Answer 1: "I would remove the AND department_id <> 90 line** from the WHERE clause."
2. **Answer 2: "Yes, the Executive department would appear in the report.**
 - Its data is: employee_count = 3, average_salary = 19,333.
 - We check the first HAVING condition: (`COUNT(*) < 5 AND AVG(salary) > 8000`).
 - Is `3 < 5`? Yes.
 - Is `19333 > 8000`? Yes.
 - Since both parts of the AND condition are true, the department meets the criteria and would be included in the results."

DAY 3

Objectives: By the end of this day, students will be able to:

- Modify and delete data safely using UPDATE and DELETE.
 - Understand the role of basic data types.
 - Simplify complex queries with Common Table Expressions (CTEs).
 - Perform advanced analysis using analytic functions with PARTITION BY.
 - Complete a final challenge integrating all learned concepts.
-

Block 1: Data Manipulation (UPDATE and DELETE) (25 minutes)

Concept: Completing our set of Data Manipulation Language (DML) commands. The WHERE clause is **CRITICAL** for these commands to prevent accidental mass data changes.

Structures and Practical Activities:

1. **The UPDATE Statement:** Modifies existing records.

- **Structure:** UPDATE table_name SET column1 = value1, ... WHERE condition;
- **Task:** The employee 'Maria Souza' (employee_id 300) we hired on Day 1 is getting a 5% salary raise.

```
UPDATE employees SET salary = salary * 1.05 WHERE employee_id = 300;
```

```
-- Verification (always verify your changes!)

SELECT first_name, salary FROM employees WHERE employee_id = 300;
```

2. **The DELETE Statement:** Removes existing records.

- **Structure:** DELETE FROM table_name WHERE condition;
 - **Task:** The temporary employee 'Fabio Martins' (employee_id 301) has finished his contract. Remove him from the system.

```
DELETE FROM employees WHERE employee_id = 301;
```

```
-- Verification

SELECT * FROM employees WHERE employee_id = 301; -- Should return no rows
```
 - **CRITICAL WARNING:** "What happens if you forget the WHERE clause in a DELETE statement? You delete **every single row** in the table. Always be careful."
-

Block 2: A Quick Look at Data Types (15 minutes)

Concept: Every column in a table has a data type that defines what kind of data it can store. This is fundamental to database design and function behavior.

Most Common Types in the HR Schema:

- VARCHAR2(size): For variable-length text strings, like first_name or email. The (size) determines the maximum length.
- NUMBER(precision, scale): For numeric data. salary is a NUMBER. precision is the total number of digits, scale is the number of digits after the decimal point.
- DATE: For storing date and time values, like hire_date.

Activity:

- Ask students to describe the employees table using the DESC command and identify the data type for employee_id, salary, and hire_date.
 - DESC employees;
 - This is a brief, conceptual block. The goal is awareness, not deep technical detail.
-

Block 3: Simplifying Queries with CTEs (Common Table Expressions) (35 minutes)

Concept: A CTE allows you to define a temporary, named result set that you can reference within your main query. It's like creating a temporary view to make your code more readable and organized.

Structure & Practical Activity:

- **Structure:** WITH cte_name AS (...your subquery...) SELECT ... FROM cte_name;
- **Task:** Let's create a report of average salaries, but only for departments in the 'Americas' region. We can use a CTE to first find those departments.

```
WITH AmericasDepartments AS ( -- This CTE finds all department IDs located in the Americas
  SELECT d.department_id
  FROM departments d
  JOIN locations l ON d.location_id = l.location_id
  JOIN countries c ON l.country_id = c.country_id
  JOIN regions r ON c.region_id = r.region_id
  WHERE r.region_name = 'Americas'
)-- Now, use the CTE in the main query
```

```
SELECT d.department_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.department_id IN (SELECT department_id FROM AmericasDepartments) -- Filter using the CTE
GROUP BY d.department_name;
```

- **Key Takeaway:** The CTE breaks a complex problem into a logical, readable first step.
-

Block 4: Analytic Functions (PARTITION BY) (40 minutes)

Concept: Aggregate functions (GROUP BY) collapse rows. Analytic functions compute a value over a group of rows (a "partition") but return a value **for each row**.

Structure & Practical Activity:

- **Task:** Rank employees within each department based on their salary, from highest to lowest.
 - **Structure:** FUNCTION() OVER (PARTITION BY column_to_group ORDER BY column_to_sort)

```
SELECT first_name, last_name, department_id, salary, RANK() OVER (PARTITION BY department_id
  ORDER BY salary DESC) AS salary_rank_in_dept
FROM employees
WHERE department_id IS NOT NULL;
```

Exercise: For each employee, show their salary *and* the average salary of their specific department on the same row. This is impossible with GROUP BY but easy with analytic functions.

```
SELECT first_name, salary, department_id, AVG(salary) OVER (PARTITION BY department_id) AS
  avg_dept_salary
FROM employees;
```

Test

Scenario: The company is implementing an "Employee of the Quarter" award. To be eligible, an employee must be one of the top 3 earners within their respective department. The final report should only list these top performers but must exclude anyone from the 'Sales' or 'Marketing' departments, as they have a separate commission-based bonus program.

Your Task: Write a single query that produces the final eligibility list.

Requirements:

1. Identify the top 3 employees by salary within each department. Use a ranking function.
2. The report must display the employee's first_name, their department_name, their job_title, their actual salary, and their rank within the department.
3. Completely exclude employees from the 'Sales' (dept ID 80) and 'Marketing' (dept ID 20) departments.
4. The final output must be ordered by department name and then by rank.
5. **Hint:** This is difficult to do in one step. Consider using a CTE to perform the initial ranking, and then select from the CTE in your final query to apply the rank filter (WHERE rank <= 3).

Solution (for the instructor):

WITH RankedEmployees AS (-- Step 1: Rank all employees within their department

```
SELECT  e.first_name,  d.department_name,  j.job_title,  e.salary,  e.department_id,
        RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS salary_rank
FROM    employees e
JOIN    departments d ON e.department_id = d.department_id
JOIN    jobs j ON e.job_id = j.job_id
```

)-- Step 2: Select from the ranked list and apply the final filters

```
SELECT first_name, department_name, job_title, salary, salary_rank
FROM   RankedEmployees
WHERE  salary_rank <= 3 -- Identify the top 3 performers  AND
       department_id NOT IN (20, 80) -- Exclude Sales and Marketing
ORDER BY department_name, salary_rank;
```

This final test forces students to use a CTE, a JOIN, an Analytic Function with PARTITION BY, and WHERE filtering on both regular columns and the calculated rank, thus testing nearly every concept from all three days.